

Then we sample from this distribution to get an action, such that the most probable action (the "best" action) is most likely to be sampled. At the end of the episode, we compute the *return* of the episode, which is essentially the sum of the discounted rewards in the episode. The return is calculated as

$$R = \sum_t \gamma_t \cdot r_t$$

After the game is over, the return for that episode is the sum of all the rewards acquired, multiplied by their respective discount rate, where γ_t exponentially decays as a function of time. For example, if action 1 was taken in state *A* and resulted in a return of +10, the probability of action 1 given state *A* will be increased a little, whereas if action 2 was taken in state *A* and resulted in a return of –20, the probability of action 2 given state *A* will decrease. Essentially, we minimize this loss function:

$$\text{Loss} = -log(P(a|S)) \times R$$

This says "minimize the logarithm of the probability of the action *a* given state *S* times the return *R*." If the reward is a big positive number, and $P(a_1|S_A) = 0.5$ for example, minimizing this loss will involve increasing this probability. So with REINFORCE we just keep sampling episodes (or trajectories more generally) from the agent and environment, and periodically update the policy parameters by minimizing this loss.

Note

Remember, we only apply the logarithm to the probability because a probability is bounded by 0 and 1, whereas the log probability is bounded by –∞ (negative infinity) and 0. Given that numbers are represented by a finite number of bits, we can represent very small (close to 0) or very large